



Ciclo Formativo de Grado Superior

Desarrollo de Aplicaciones Multiplataforma
(2º Curso)



Sistema de gestión de punto de venta

aplicación web full-stack para la gestión integral de
establecimientos de hostelería

NOMBRE: JESÚS

APELLIDOS: DELGADO SÁNCHEZ

1. INDICE



1. INDICE

2. INTRODUCCIÓN

3. REQUISITOS PARA EL DESARROLLO

3.1. REQUISITOS PRINCIPALES

3.2. REQUISITOS OPCIONALES

3.3. DEPENDENCIAS ENTRE SERVICIOS

4. PUESTA EN MARCHA Y USO

4.1. ESTRUCTURA DEL CÓDIGO Y ARCHIVOS

4.2. DESCRIPCIÓN DE LAS PANTALLAS Y FUNCIONALIDADES

4.2.1. PANEL DE CONTROL (DASHBOARD) - BARRA LATERAL Y NAVEGACIÓN

4.2.2. INICIO - GESTIÓN DE CAJA Y ESTADÍSTICAS DEL TURNO

4.2.3. GESTIÓN DE MESAS EN TIEMPO REAL

4.2.4. CARTA - CATÁLOGO DE PRODUCTOS

4.2.5. EMPLEADOS - GESTIÓN DE USUARIOS

4.2.6. REPORTES - ESTADÍSTICAS GLOBALES

4.2.7. CONFIGURACIÓN DEL LOCAL

4.2.8. TERMINAL DEL CAMARERO (TPV)

4.2.9. VISOR DE COCINA (KDS)

4.2.10. MENÚ DEL CLIENTE (ACCESO POR CÓDIGO QR)

4.3. CONCEPTOS CLAVE Y TECNOLOGÍAS

4.4. BASE DE DATOS (ESQUEMA COMPLETO)

4.5. PROCESOS PRINCIPALES

5. POSIBLES AMPLIACIONES

6. VALORACIÓN PERSONAL

7. APÉNDICES

2. INTRODUCCIÓN



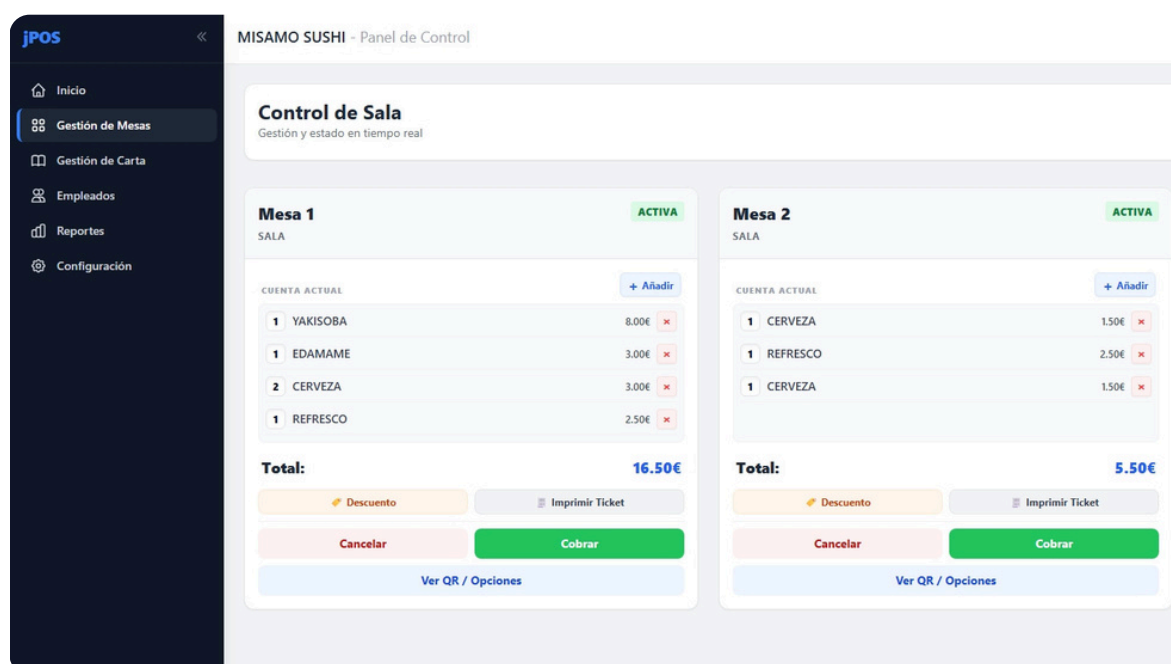
jPOS es un sistema de punto de venta (TPV) completo orientado a establecimientos de hostelería (restaurantes, bares, cafeterías...), desarrollado como Proyecto de Fin de Grado del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma.

El proyecto nace de la necesidad de digitalizar la operativa de un restaurante que aún no está digitalizado de forma íntegra: desde que el cliente llega y escanea el código QR de su mesa, hasta que el personal de cocina prepara el pedido, el camarero lo sirve y el encargado realiza el cobro y cuadra la caja al final del turno.

A diferencia de los TPV tradicionales, jPOS funciona íntegramente en el navegador y en tiempo real gracias a WebSockets: cualquier cambio en un pedido se refleja instantáneamente en todas las pantallas conectadas (mesas, cocina, terminal camarero, menu cliente) sin necesidad de recargar la página.

El sistema se compone de dos proyectos independientes que trabajan en conjunto:

- **Backend (jpos-backend):** API REST y servidor WebSocket desarrollado con NestJS 11 sobre Node.js, con Prisma ORM como capa de acceso a la base de datos PostgreSQL.
- **Frontend (jpos-frontend):** Aplicación web SPA desarrollada con Angular 21, diseñada con TailwindCSS y comunicada con el servidor mediante HTTP y Socket.IO.



3. REQUISITOS PARA EL DESAROLLO



3.1. Requisitos principales

Node.js	v18 o superior (recomendado v22 LTS)
npm	v9 o superior (incluido con Node.js)
PostgreSQL	v14 o superior, en ejecución en el puerto 5432
Sistema operativo	Windows 10/11, macOS 12+ o Linux (Ubuntu 22.04+)
Navegador	Google Chrome, Mozilla Firefox o Microsoft Edge (versión reciente)
RAM mínima	4 GB (se recomiendan 8 GB para desarrollo cómodo)
Espacio en disco	~500 MB libres (node_modules incluidos)
Red local	Red Wi-Fi para probar el menú QR desde dispositivos móviles

3.2. Requisitos opcionales

- **Prisma Studio:** interfaz visual de base de datos integrada en Prisma (npx prisma studio). Útil para inspeccionar datos durante el desarrollo.
- **Postman o Bruno:** clientes REST para probar los endpoints de la API de forma manual.
- **Un dispositivo móvil en la misma red:** para simular el flujo completo del cliente escaneando el QR y realizando un pedido.
- **Docker:** para ejecutar PostgreSQL en un contenedor sin instalación local.

3.3. Dependencias entre servicios

El orden de arranque es crítico para el correcto funcionamiento del sistema:

- **1º** — PostgreSQL debe estar en ejecución antes de arrancar el backend.
- **2º** — El backend (NestJS, puerto 3000) debe estar activo antes que el frontend.
- **3º** — El frontend (Angular, puerto 4200) conecta con el backend al iniciarse.
- **4º** — Los clientes (navegadores) se conectan al frontend una vez está en marcha.

4. PUESTA EN MARCHA Y USO



4.1. Estructura del código y archivos

Backend (Node.js/NestJS) — [jpos-backend/](#)

src/auth/	Módulo de autenticación: login, guards JWT, interceptor de tokens, cifrado bcrypt de contraseñas.
src/mesas/	Gestión de mesas: CRUD, estados (INACTIVA/ACTIVA/OCUPADA/PAGANDO), generación de URL para QR, apertura/cierre de sesión de mesa.
src/pedidos/	Pedidos: CRUD, lógica de comanda, descuentos, cobro, cancelación y el WebSocket Gateway.
src/carta/	Catálogo: CRUD de categorías y productos con soporte bilingüe (ES/EN).
src/caja/	Turnos de caja: apertura con fondo inicial, cierre con arqueo, estadísticas del turno.
src/reportes/	Informes de ventas: filtros por fecha, productos más vendidos, facturación total.
src/usuarios/	Gestión de empleados: CRUD, activación/desactivación, asignación de roles.

src/configuracion/	Datos del negocio: nombre, NIF, dirección, IVA por defecto, moneda, idioma, horarios.
src/prisma/	Servicio singleton de acceso a base de datos (PrismaService) compartido por todos los módulos.
prisma/schema.prisma	Definición completa del esquema de la base de datos con todos los modelos y relaciones.
prisma/migrations/	Historial de migraciones SQL que documentan la evolución del esquema.
prisma/seed.ts	Script de datos iniciales: categorías de ejemplo, productos y usuario administrador.
.env	Variables de entorno: DATABASE_URL, JWT_SECRET y puerto del servidor.

Frontend (Angular 21) — [jpos-frontend/](#)

src/app/features/auth/	Pantalla de login con validación, opción “Recuérdame” (token 7d vs 24h).
src/app/features/cliente/	Menú público: redirección QR (qr-redirect) y vista de carta del cliente (menu-cliente).
src/app/features/cocina/	Visor KDS (Kitchen Display System) para que la cocina gestione los pedidos entrantes.
src/app/features/pos/terminal/	Terminal TPV del camarero: vista de mesas + carta para añadir productos.
src/app/features/dashboard/	Panel de control: inicio (caja), mesas, carta, empleados, reportes, configuración.
src/app/core/services/	Servicios Angular: auth, socket, caja, carta, mesa, reportes, configuración, notificaciones.

src/app/core/guards/	Guards de ruta: authGuard (requiere login) y rolGuard (requiere rol mínimo).
src/app/layout/	DashboardLayout: sidebar colapsable, topbar con nombre del local y toast de notificaciones.
public/assets/i18n/	Archivos de traducción: es.json (español) y en.json (inglés) para toda la interfaz.

4.2. Descripción de las pantallas y funcionalidades

4.2.1. Panel de control (Dashboard) — Barra lateral y navegación

El panel de control es la interfaz principal del personal del restaurante. Se accede tras el login y está protegido por authGuard. Consta de un layout fijo con dos zonas: una barra lateral de navegación (sidebar) y el área de contenido principal que cambia según la sección activa.

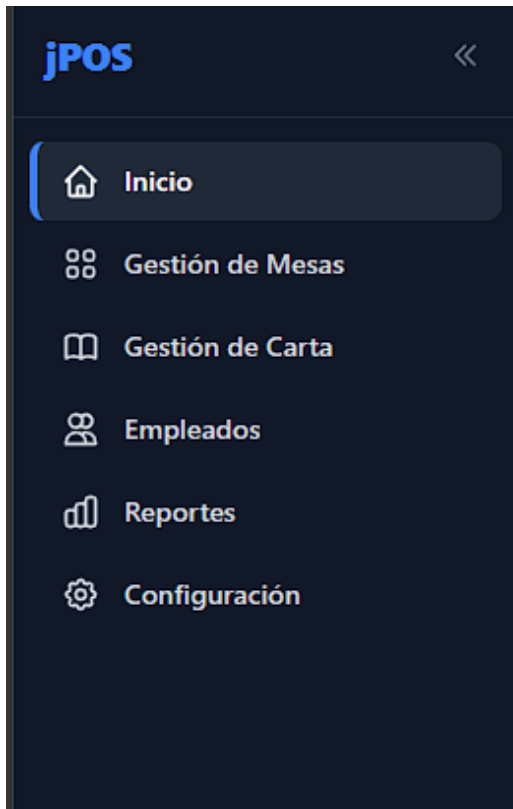
La barra lateral incluye las siguientes secciones:

Sección	Ruta	Acceso por rol	Descripción
Inicio (Estado de caja)	/dashboard	Todos los roles	Gestión del turno de caja, estadísticas y registro de operaciones del turno actual.
Mesas	/mesas	ADMINISTRADOR, JEFE, EMPLEADO	Control de sala en tiempo real: pedidos activos, cobro, descuentos, impresión de ticket.
Carta	/carta	ADMINISTRADOR, JEFE	Gestión del catálogo: categorías y productos con precios, IVA e imágenes.

Empleados	/empleados	ADMINISTRADOR	Alta, modificación y activación/desactivación de empleados. Asignación de roles.
Reportes	/reportes	ADMINISTRADOR	Estadísticas de ventas filtrables por fecha, gráficas y productos más vendidos.
Configuración	/configuracion	ADMINISTRADOR	Datos del negocio: nombre, NIF, IVA, moneda, idioma y horarios del local.
Cerrar sesión	—	Todos los roles	Cierra la sesión del usuario y redirige al login.

Características adicionales del layout:

- La barra lateral es colapsable en escritorio: puede reducirse a solo iconos para ganar espacio de trabajo.
- En dispositivos móviles el sidebar se oculta y se muestra mediante un botón hamburguesa en la barra superior.
- La topbar muestra el nombre del local (extraído de la configuración) y el texto “Panel de Control”.
- Se muestra un badge rojo con contador animado sobre el icono de Gestión de Mesas cuando hay mesas que han solicitado la cuenta, visible incluso con el sidebar colapsado.
- Un toast de notificación emergente en la esquina superior derecha avisa en tiempo real cuando una mesa solicita la cuenta, indicando el número de mesa.



Barra lateral del dashboard (expandida y colapsada)



Menú hamburguesa en móvil: Al pulsar en él se mostrarán las mismas secciones de la barra lateral

4.2.2. Inicio — Gestión de caja y estadísticas del turno

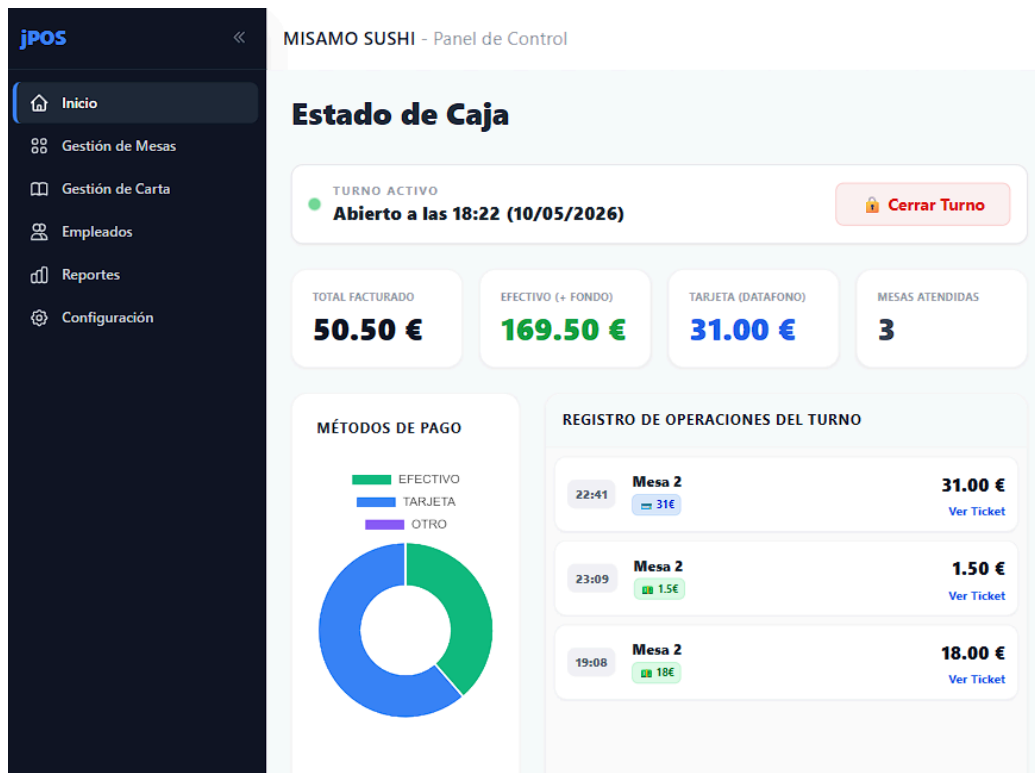
La pantalla de inicio es el centro de control económico del restaurante. Su comportamiento varía según el estado de la caja:

Estado CERRADA:

- Muestra un reloj digital en tiempo real con la fecha y hora actuales.
- Permite configurar el fondo inicial de caja (importe en efectivo).
- Botón para abrir el turno de caja.

Estado ABIERTA:

- Indicador de turno activo con la hora de apertura y botón para cerrar el turno.
- 4 tarjetas de estadísticas en tiempo real: Total facturado, Efectivo en cajón (fondo + cobros en efectivo), Total tarjeta/datáfono y Número de mesas atendidas.
- Gráfico de donut con la distribución de métodos de pago: efectivo, tarjeta y otro.
- Registro de operaciones del turno: lista de todos los pedidos cobrados con hora, número de mesa, métodos de pago usados e importe. Cada entrada permite ver el ticket completo.
- Modal de cierre de turno con cuadro de caja: muestra el fondo inicial, total por tarjeta y el efectivo esperado en el cajón antes de confirmar el cierre.
- La pantalla se actualiza automáticamente vía WebSocket cuando se cobra una mesa (evento **mesas_actualizadas**).



Pantalla de inicio con caja abierta y estadísticas del turno

Cuadre de Caja

Revisa los importes antes de cerrar el turno

Fondo inicial puesto:	150.00 €
Total cobrado en Tarjeta (Datafono):	31.00 €
Efectivo total que debe haber en el cajón:	169.50 €

Si los números cuadran, retira el efectivo y confirma el cierre.

Cancelar Confirmar Cierre

Modal de cierre de turno con cuadro de caja

4.2.3. Gestión de mesas en tiempo real

La pantalla de mesas es el corazón operativo del restaurante. Se actualiza en tiempo real mediante WebSockets y permite gestionar toda la actividad de la sala desde una sola pantalla.

Características de visualización:

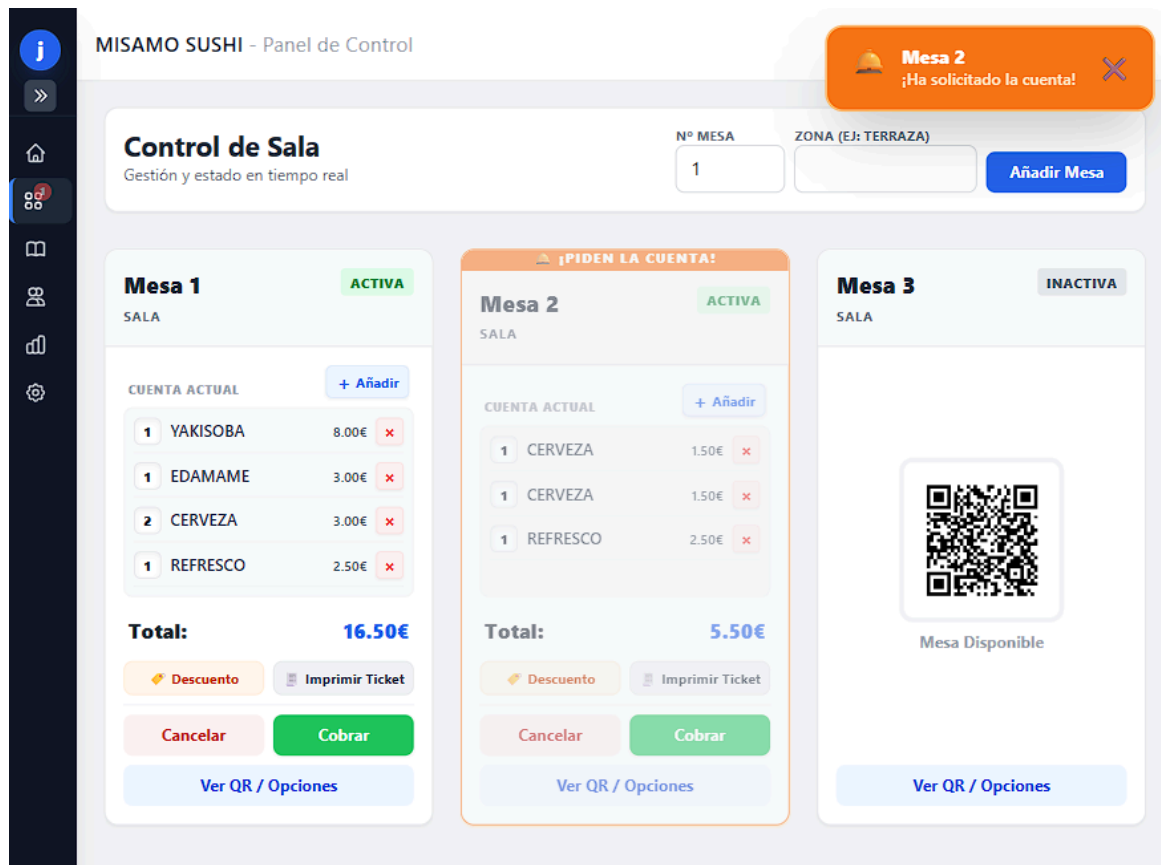
- Cada mesa se muestra como una tarjeta. Las mesas inactivas muestran su código QR, que el cliente pueda escanear para realizar pedidos desde su móvil.
- Las mesas activas muestran el desglose del pedido en curso línea a línea con cantidades y precios.
- Las tarjetas de mesas que han solicitado la cuenta parpadean con borde naranja y muestran un banner animado (“¡Piden la cuenta!”) para llamar la atención del camarero.
- El estado de la sala se actualiza automáticamente al recibir los eventos WebSocket: **pedido_actualizado**, **mesas_actualizadas**, **cuenta_cerrada** y **cuenta_solicitada**.

Acciones disponibles por mesa (con pedido activo):

- Agregar producto: abre un modal con la carta completa por categorías para añadir productos directamente al pedido activo de la mesa (mini-TPV integrado).
- Quitar línea de pedido: elimina un producto del pedido (solo JEFE y ADMINISTRADOR).
- Descuento: aplica un descuento al total del pedido, ya sea en porcentaje o en importe fijo (solo JEFE y ADMINISTRADOR). Se comunica al servidor vía WebSocket (evento **aplicar_descuento**).
- Imprimir ticket: genera una vista de impresión del navegador con el desglose del pedido para entregarlo al cliente antes del cobro.
- Cancelar: cancela el pedido activo de la mesa y la devuelve a estado inactivo (solo JEFE y ADMINISTRADOR).
- Cobrar: abre el modal de cobro donde se introduce el importe pagado por cada método (efectivo, tarjeta, otro), permite pagos mixtos, y al confirmar cierra el pedido y libera la mesa.

Administración de mesas:

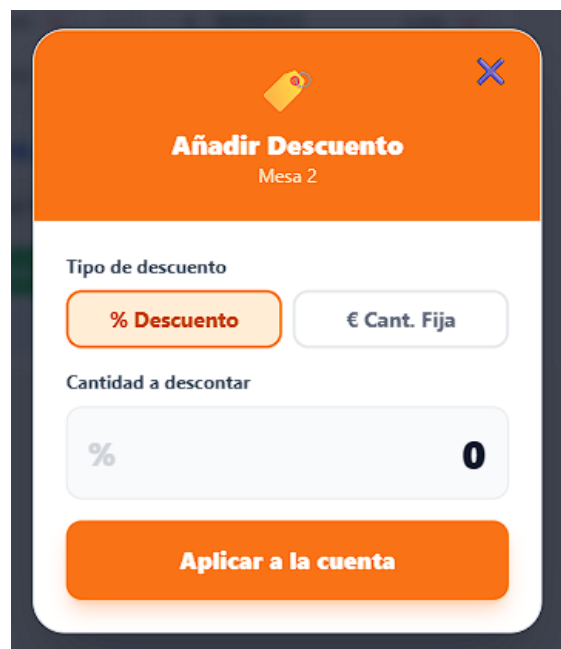
- Crear nuevas mesas indicando número y zona (ej: Terraza, Interior, Barra).
- Editar datos de una mesa existente.
- Ver y descargar/imprimir el QR de cada mesa.



Grid de mesas con diferentes estados (activa, pidiendo cuenta, inactiva)



Modal de cobro

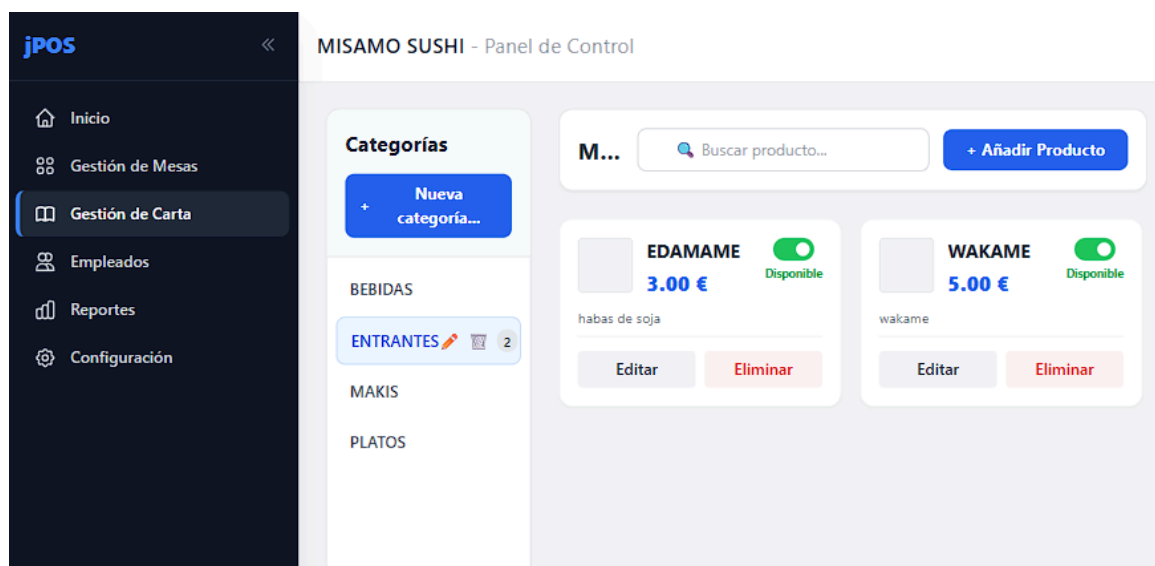


Modal de descuento

4.2.4. Carta — Catálogo de productos

Permite gestionar el catálogo completo del restaurante. Accesible solo para JEFE y ADMINISTRADOR.

- Creación y edición de categorías (Entrantes, Platos, Postres, Bebidas...) con nombre en español e inglés y orden de aparición.
- Creación y edición de productos con elección de categoría, nombre en español e inglés, descripción, precio e imagen.
- Marcar productos como no disponibles: se muestran en gris en el menú del cliente y en el KDS, y se notifica a todos los dispositivos via WebSocket (evento **carta_actualizada**).



Pantalla de gestión de carta con categorías y productos

4.2.5. Empleados — Gestión de usuarios

Panel exclusivo del ADMINISTRADOR para gestionar el equipo del restaurante.

- Alta de nuevos empleados con nombre, correo, contraseña y rol (ADMINISTRADOR, JEFE, EMPLEADO).
- Edición de datos de empleados existentes.
- Activación y desactivación de empleados: los empleados desactivados no pueden iniciar sesión.

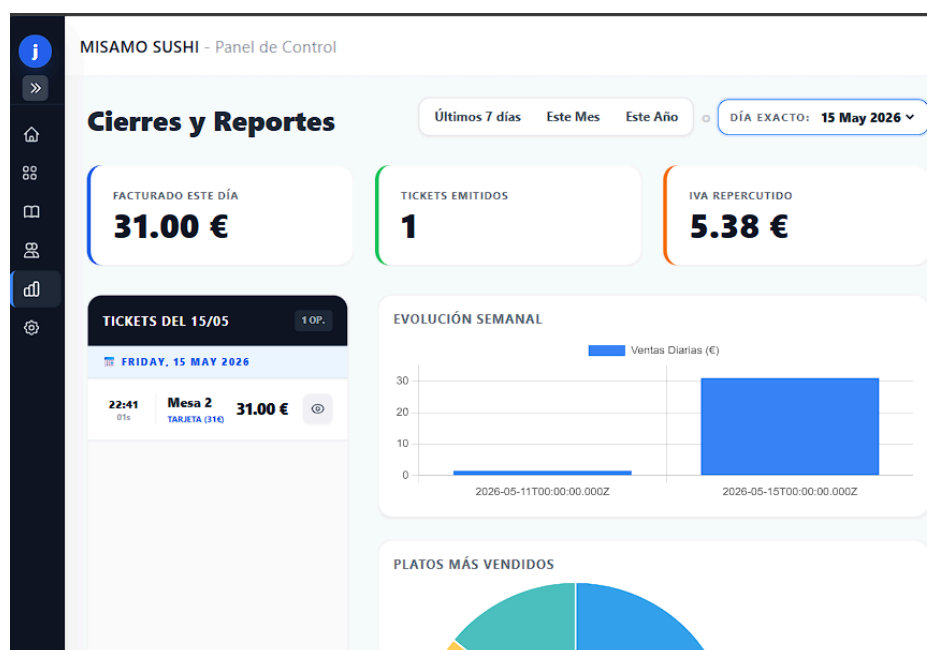


Listado de empleados con roles y estado (activo/inactivo)

4.2.6. Reportes — Estadísticas globales

Sección de análisis económico disponible solo para el ADMINISTRADOR.

- Filtros de fecha para consultar periodos específicos.
- Total facturado, número de pedidos y ticket medio en el periodo seleccionado.
- Desglose por método de pago (efectivo, tarjeta, otro).
- Ranking de productos más vendidos.
- Gráficas de evolución de ventas (Chart.js / ng2-charts).



Pantalla de reportes con gráficos y estadísticas

4.2.7. Configuración del local

Permite al ADMINISTRADOR personalizar los parámetros globales del sistema.

- Nombre del restaurante (aparece en la topbar y en los tickets).
- NIF / CIF del negocio.
- Dirección y teléfono de contacto.
- IVA por defecto aplicado a los nuevos productos.
- Moneda (símbolo mostrado en precios y tickets).
- Idioma del sistema: español o inglés (afecta a toda la interfaz vía ngx-translate).
- Mensaje de pie de ticket.
- Horarios del local en formato JSON (por día de la semana con apertura y cierre).

MISAMO SUSHI - Panel de Control

Ajustes del Sistema

Preferencias Globales

Idioma del Sistema: **ES Español** (dropdown menu)

IVA por defecto (%): **10**

El sistema ahora traducirá la interfaz automáticamente.

Datos de Facturación y Ticket

Estos datos aparecerán impresos en los recibos de los clientes.

Nombre Comercial del Local: **MISAMO SUSHI**

NIF / CIF: **B-12345678**

Teléfono de Contacto: **955000000**

Dirección Completa: **Avda. de los Majuelos, 1 41960 Gines (Sevilla)**

Guardar Cambios

Pantalla de configuración del local

4.2.8. Terminal del camarero (TPV)

Pantalla independiente del dashboard, diseñada para ser usada en una tablet o pantalla secundaria por el personal de sala. Accesible para todos los roles.

Vista de mesas:

- Muestra todas las mesas del restaurante con su estado actual y el número de líneas de pedido pendientes.
- Actualizada en tiempo real vía WebSocket (**pedido_actualizado**, **mesas_actualizadas**).
- Permite seleccionar una mesa para pasar a la vista TPV.

Vista TPV (con mesa seleccionada):

- Muestra la carta completa dividida por categorías para añadir productos al pedido de la mesa.
- Muestra el ticket acumulado de la mesa con cantidades y subtotales.
- Botón para enviar la comanda a cocina vía WebSocket (evento **enviar_comanda**), que actualiza instantáneamente el KDS.
- El camarero puede solicitar la cuenta desde el terminal (igual que el cliente desde el menú QR), disparando el evento **solicitar_cuenta**.
- Recibe notificaciones sonoras cuando un plato está listo en cocina (evento **plato_listo_notificacion**).
- Permite marcar platos como entregados (evento **marcar_entregado_terminal**) una vez servidos al cliente.

Nota importante: el terminal NO permite cobrar directamente. El cobro se realiza exclusivamente desde la pantalla de Mesas del dashboard, garantizando que quede registrado en la caja del turno activo.



Terminal del camarero - vista de mesas



Terminal del camarero - vista de ticket y carta

4.2.9. Visor de cocina (KDS)

Pantalla independiente para el personal de cocina. Muestra en tiempo real todos los pedidos que deben prepararse. Accesible para todos los roles.

- Recibe instantáneamente los nuevos pedidos enviados desde el terminal o el menú QR del cliente (evento **nuevo_pedido_cocina**).
- Muestra el número de mesa, los productos y las notas especiales de cada línea.
- El cocinero puede marcar cada línea de pedido como lista (evento **marcar_linea_kds**), lo que actualiza el estado en todas las pantallas y dispara una notificación sonora en el terminal del camarero.
- Permite marcar productos como no disponibles directamente desde cocina (evento **cambiar_disponibilidad_kds**), actualizando el menú del cliente en tiempo real.



Visor KDS de cocina con pedidos pendientes

4.2.10. Menú del cliente (acceso por código QR)

Interfaz pública (sin autenticación) accesible desde el móvil del cliente al escanear el QR de su mesa.

- El QR redirige a **/qr/:id**, que valida el identificador de la mesa, crea una sesión con token único y redirige al menú.
- El menú muestra todos los productos disponibles agrupados por categorías, con nombre, descripción, precio e imagen.
- Los productos no disponibles se muestran en gris y no pueden añadirse al carrito.
- El cliente puede añadir productos al carrito, ajustar cantidades y añadir notas por línea.
- Al confirmar, el pedido se envía vía WebSocket (evento **enviar_comanda**) y llega instantáneamente a cocina y terminal.
- El cliente puede solicitar la cuenta desde el menú con el botón correspondiente (evento **solicitar_cuenta**).
- El idioma del menú se ajusta automáticamente al configurado en el panel (ES/EN).



Menú del cliente - cuenta actual



Menú del cliente - carta para pedir

4.3. Conceptos clave y tecnologías

Tecnología	Versión	Uso en el proyecto
NestJS	11.x	Framework backend. Arquitectura modular con controladores, servicios e inyección de dependencias. Simplifica la creación de la API REST y los WebSockets.
Prisma ORM	6.x	Capa de acceso a base de datos type-safe. El esquema define todos los modelos; Prisma genera el cliente TS y gestiona las migraciones.
PostgreSQL	14+	Base de datos relacional principal. Almacena usuarios, mesas, pedidos, carta, configuración y turnos de caja.
JWT (JSON Web Token)	—	Autenticación sin estado. El servidor firma un token con los datos del usuario (id, correo, rol). El cliente lo envía en cada petición HTTP mediante la cabecera Authorization: Bearer.
bcrypt	6.x	Hash seguro de contraseñas. Las contraseñas nunca se almacenan en texto plano en la base de datos.
Socket.IO / WebSockets	4.x	Comunicación bidireccional en tiempo real. Permite que los pedidos del cliente, los cambios de estado de cocina y los avisos de cuenta se reflejen instantáneamente en todas las pantallas.
Angular	21.x	Framework SPA frontend. Usa componentes standalone, el nuevo sistema de control de flujo (@if, @for), signals y lazy loading.
TailwindCSS	3.x	Framework CSS de utilidades. Toda la interfaz se construye directamente en el HTML sin hojas de estilo personalizadas, con diseño responsivo incluido.
ngx-translate	17.x	Internacionalización del frontend. Carga los archivos es.json / en.json y sustituye las claves de traducción en tiempo de ejecución.

angularx-qrcode	21.x	Generación de códigos QR en el frontend para cada mesa. El QR codifica la URL pública del menú del cliente.
Chart.js / ng2-charts	4.x / 10.x	Gráficos interactivos (dona y líneas) en la pantalla de reportes e inicio del dashboard.
RxJS	7.x	Programación reactiva en Angular. Los servicios devuelven Observables; los WebSockets se consumen como streams reactivos.

Roles y control de acceso

Rol	Secciones accesibles	Acciones especiales
ADMINISTRADOR	Todas las secciones	Acceso total: administración de empleados, reportes, configuración, descuentos, cancelaciones.
JEFE	Dashboard (sin Reportes / Config / Empleados), Terminal, KDS	Puede aplicar descuentos y cancelar pedidos. No accede a reportes ni configuración.
EMPLEADO	Mesas (solo lectura de pedidos), Terminal, KDS	Puede añadir productos y solicitar cuenta. No puede cancelar ni aplicar descuentos.

4.4. Base de datos (esquema completo)

La base de datos PostgreSQL se gestiona íntegramente con Prisma. A continuación se describen todas las entidades y relaciones del esquema:

- Categoría > Producto (una categoría tiene muchos productos)
- Producto > LineaPedido (un producto puede aparecer en muchas líneas de pedido)
- Mesa > Pedido (una mesa genera muchos pedidos a lo largo del tiempo)
- Mesa <> Sesión (una mesa tiene como máximo una sesión activa, relación opcional en ambos sentidos)
- Sesión > Pedido (una sesión agrupa varios pedidos)
- Pedido > LineaPedido (un pedido contiene muchas líneas)

Configuracion y TurnoCaja son entidades independientes sin relaciones directas con el resto del esquema, se consultan de forma autónoma.

Enums Entidades principales Entidades de soporte



Usuario

Campo	Tipo	Descripción
id	String (UUID)	Identificador único generado automáticamente.
nombre	String	Nombre completo del empleado.
correo	String (unique)	Dirección de correo electrónico. Debe ser única. Se usa como credencial de login.
contrasena	String	Contraseña cifrada con bcrypt. Nunca se almacena en texto plano.
rol	Enum (Rol)	Rol del usuario: ADMINISTRADOR, JEFE o EMPLEADO.
activo	Boolean	Indica si el usuario puede iniciar sesión. Por defecto true.
creadoEn	DateTime	Fecha y hora de creación del registro.

Mesa

Campo	Tipo	Descripción
id	String (UUID)	Identificador único.
numero	Int (unique)	Número de mesa. Único en el restaurante.
zona	String?	Zona del restaurante (Terraza, Interior, Barra...). Opcional.
qrUrl	String	URL codificada en el QR que redirige al menú del cliente para esta mesa.
estado	Enum (MesaEstado)	Estado: INACTIVA (sin clientes), ACTIVA (con pedido), OCUPADA, PAGANDO.

sesionActualId	String? (unique)	Referencia a la sesión activa de la mesa. Null si no hay clientes.
cuentaSolicitada	Boolean	True cuando el cliente ha pulsado “Solicitar cuenta”. Activa el aviso visual.

Sesión

Campo	Tipo	Descripción
id	String (UUID)	Identificador único.
abiertaEn	DateTime	Momento en que el primer cliente escaneó el QR.
cerradaEn	DateTime?	Momento en que se cobró o canceló la cuenta. Null si está activa.
tokenAcceso	String (unique)	Token único generado al abrir la sesión. Permite al cliente hacer pedidos sin login.

Pedido

Campo	Tipo	Descripción
id	String (UUID)	Identificador único.
estado	Enum (PedidoEstado)	Estado: PENDIENTE, EN_COCINA, LISTO, ENTREGADO, PAGADO, CANCELADO.
total	Decimal(10,2)	Importe total del pedido (actualizado dinámicamente).
llevar	Boolean	Indica si el pedido es para llevar. Por defecto false.
creadoEn	DateTime	Fecha y hora de creación del pedido.

actualizadoEn	DateTime	Fecha y hora de la última modificación (automático con @updatedAt).
pagos	Json?	Desglose del cobro: { efectivo: X, tarjeta: Y, otro: Z }. Permite pagos mixtos.
mesald	String	Referencia a la mesa a la que pertenece el pedido.
sesionId	String?	Referencia a la sesión activa de la mesa.

Línea de pedido (LineaPedido)

Campo	Tipo	Descripción
id	String (UUID)	Identificador único.
cantidad	Int	Número de unidades del producto en esta línea.
notas	String?	Notas o peticiones especiales del cliente para este plato (sin gluten, sin sal...).
precioUnitario	Decimal(10,2)	Precio del producto en el momento del pedido. Snapshot para evitar que cambios futuros de precio afecten pedidos ya realizados.
listo	Boolean	True cuando la cocina marca el plato como preparado desde el KDS.
entregado	Boolean	True cuando el camarero marca el plato como servido al cliente.
pedidoId	String	Referencia al pedido al que pertenece esta línea.
productId	String	Referencia al producto de la carta.

Producto

Campo	Tipo	Descripción
id	String (UUID)	Identificador único.
nombre_es	String	Nombre del producto en español (requerido).
nombre_en	String?	Nombre del producto en inglés (opcional).
descripcion_es	String?	Descripción en español (opcional).
descripcion_en	String?	Descripción en inglés (opcional).
precio	Decimal(10,2)	Precio de venta al público (con IVA incluido).
imagenUrl	String?	URL de la imagen del producto. Opcional.
disponible	Boolean	Si es false, el producto aparece en gris y no puede pedirse.
iva	Decimal(5,2)	Porcentaje de IVA aplicado al producto.
categoriald	String	Referencia a la categoría a la que pertenece.

Categoría

Campo	Tipo	Descripción
id	String (UUID)	Identificador único.
nombre_es	String	Nombre de la categoría en español.
nombre_en	String?	Nombre de la categoría en inglés (opcional).
orden	Int	Posición de la categoría en la carta. Permite ordenar (Entrantes antes que Postres).

Configuración

Campo	Tipo	Descripción
id	Int (siempre = 1)	Registro único. Solo existe una fila de configuración en el sistema.
nombreLocal	String	Nombre del restaurante. Aparece en la topbar y en los tickets.
nif	String?	NIF o CIF del negocio. Opcional.
direccion	String?	Dirección postal del restaurante. Opcional.
telefono	String?	Teléfono de contacto. Opcional.
ivaDefecto	Decimal(5,2)	IVA aplicado por defecto a los nuevos productos.
moneda	String	Símbolo de moneda mostrado en la interfaz. Por defecto “€”.
idioma	String	Código de idioma de la interfaz: “es” o “en”. Por defecto “es”.
mensajeTicket	String?	Texto del pie de ticket (ej: “¡Gracias por su visita!”).
horarios	Json?	Horarios del local en formato JSON por día de la semana.

Turno de Caja (TurnoCaja)

Campo	Tipo	Descripción
id	Int (autoincrement)	Identificador único del turno.
abiertoEn	DateTime	Fecha y hora de apertura del turno.
cerradoEn	DateTime?	Fecha y hora de cierre. Null si el turno está abierto.
fondolnicial	Decimal(8,2)	Importe colocado en caja al abrir el turno.
estado	String	Estado del turno: "ABIERTA" o "CERRADA".

```
schema.prisma U X login.html U menu-cliente.html U X visor-kds.html U
jpos-backend > prisma > schema.prisma
1 // This is your Prisma schema file,
2 // learn more about it in the docs: https://pris.ly/d/prisma-schema
3
4 // Looking for ways to speed up your queries, or scale easily with your
5 // Try Prisma Accelerate: https://pris.ly/cli/accelerate-init
6
7 generator client {
8   provider = "prisma-client-js"
9 }
10
11 datasource db {
12   provider = "postgresql"
13   url      = env("DATABASE_URL")
14 }
15
16 enum Rol {
17   ADMINISTRADOR
18   JEFE
19   EMPLEADO
20 }
21
22 model Usuario {
23   id          String  @id @default(uuid())
24   nombre      String
25   correo      String  @unique
26   contrasena  String
27   rol         Rol     @default(EMPLEADO)
28   activo      Boolean @default(true)
29   creadoEn    DateTime @default(now())
30 }
31
```

Esquema de Prisma con definición de modelos

4.5. Procesos principales

Flujo completo de un pedido (cliente QR > cocina > servicio > cobro)

Paso	Usuario	Acción	Evento WebSocket
1	Administrador	Genera el QR de la mesa desde el panel de Mesas.	—
2	Cliente	Escanea el QR con su móvil. El sistema crea una Sesión con token único.	—
3	Cliente	Selecciona productos en el menú y confirma la comanda.	enviar_comanda
4	Cocina	Recibe instantáneamente el pedido en el KDS.	nuevo_pedido_cocina
5	Camarero	Ve el pedido actualizado en el terminal.	pedido_actualizado
6	Cocina	Marca cada plato como listo conforme los prepara.	marcar_linea_kds
7	Camarero	Recibe notificación sonora. Recoge y sirve el plato. Lo marca como entregado.	marcar_entregado_terminal
8	Cliente	Solicita la cuenta desde el móvil o el camarero desde el terminal.	solicitar_cuenta
9	Dashboard	Muestra aviso de cuenta solicitada con animación en la tarjeta de la mesa.	cuenta_solicitada
10	JEFE/ADMIN	Abre el modal de cobro, introduce los importes y confirma el pago.	—
11	Sistema	Cierra el pedido (PAGADO), libera la mesa (INACTIVA) y actualiza la caja.	cuenta_cerrada, mesas_actualizadas

Flujo de autenticación

- El usuario introduce correo y contraseña en el login.
- El backend valida la contraseña con **bcrypt.compare()** y comprueba que el usuario esté activo.
- Si es correcto, genera un **JWT** firmado con el id, correo y rol del usuario. La caducidad es de 24h (o 7d con “Recuérdame”).
- El frontend almacena el token y lo inyecta automáticamente en cada petición HTTP mediante **auth.interceptor.ts**.
- **authGuard** verifica el token en cada cambio de ruta. **rolGuard** comprueba que el rol del usuario sea suficiente para la ruta solicitada.

5. POSIBLES AMPLIACIONES



- ☐ **Pedidos a domicilio y recogida en local:** módulo de pedidos online donde el cliente se registra con sus datos y puede hacer un pedido desde su casa. Nuevo rol de “Repartidor”, el cual puede asignarse pedidos. Integración con pasarela de pago para cobro previo y panel de seguimiento de estado del pedido.
- ☐ **Impresión de tickets térmicos:** integración con impresoras térmicas via protocolo ESC/POS para imprimir comandas automáticamente en cocina y tickets de cobro en la caja.
- ☐ **App móvil nativa (iOS/Android):** desarrollar la interfaz del camarero, del repartidor (módulo de pedidos online) y del cliente como app autónoma con Ionic/Capacitor, aprovechando el backend existente sin cambios.
- ☐ **Gestión de reservas:** módulo de reservas con calendario, control de aforo y notificaciones automáticas por correo o SMS al cliente.
- ☐ **Múltiples métodos de pago:** integración con pasarelas como Stripe o Redsys para permitir el pago con tarjeta directamente desde el móvil del cliente, sin necesidad de datáfono físico.

- ☐ **Estadísticas avanzadas:** análisis de productos más vendidos por categoría y franja horaria, horas pico, ticket medio por zona o mesa.
- ☐ **Modo offline y PWA:** convertir el frontend en una Progressive Web App con Service Worker para que siga funcionando durante cortes de red puntuales y sincronice cuando se recupere la conexión.
- ☐ **Gestión de inventario y proveedores:** control de stock de ingredientes, alertas de escasez y generación automática de pedidos a proveedores.
- ☐ **Multi-restaurante:** soporte para gestionar varias sucursales desde una misma instancia del sistema, con configuraciones y personal independientes por local.
- ☐ **Fidelización de clientes:** sistema de puntos o tarjeta de fidelización integrado en el menú del cliente, con histórico de visitas y descuentos personalizados.

6. VALORACIÓN PERSONAL



Origen del proyecto

La idea de desarrollar jPOS viene de una experiencia directa: durante años he trabajado en distintos restaurantes donde la digitalización del servicio era parcial o inexistente. Veía a diario cómo los camareros tomaban comandas en papel, cómo la cocina recibía tickets escritos a mano y cómo los errores o los retrasos en la comunicación entre sala y cocina eran una fuente constante de estrés y pérdida de tiempo. Esa experiencia me hizo preguntarme si, con los conocimientos adquiridos durante el ciclo, podría hacer algo que resolviera esos problemas de verdad.

Así nació jPOS: como un reto personal. Quería hacer algo que pudiera funcionar en un entorno real, que resolviera un problema real y que yo mismo hubiera necesitado en mi trabajo.

El mercado de los POS

Soy consciente de que el mercado de los sistemas POS para restaurantes está ocupado por soluciones consolidadas con años de desarrollo y aplicaciones web modernas similares. Sin embargo, creo que siempre existe espacio para entrar en un mercado cuando se ven infinidad de comercios de hostelería sin digitalizar, y sobre todo cuando se aporta una propuesta diferencial.

Los puntos fuertes en los que jPOS podría competir son varios. En primer lugar, la accesibilidad tecnológica: no requiere instalar ningún software ni adquirir hardware específico; funciona en cualquier tablet, móvil u ordenador con navegador, lo que elimina una de las principales barreras de entrada para un restaurante pequeño. En segundo lugar, el tiempo real como característica central: la sincronización instantánea es la base sobre la que está diseñado todo el sistema. En tercer lugar, la experiencia del cliente: permitir que el propio comensal haga su pedido desde el móvil escaneando el QR reduce los tiempos de espera, minimiza errores de transcripción y mejora la percepción del servicio.

Pero quizás el argumento más importante es el económico. Muchos restaurantes medianos y pequeños no pueden permitirse las cuotas mensuales de los grandes sistemas TPV del mercado, que suelen incluir hardware propietario, contratos de mantenimiento y licencias por terminal. jPOS está diseñado con la filosofía de digitalizar a precio contenido: desplegable en cualquier servidor VPS económico y sin costes de licencia por dispositivo. El objetivo sería llegar a ese segmento de restauración que quiere modernizarse pero no puede asumir grandes inversiones iniciales.

Por qué tecnología web y este stack

La aplicación debía funcionar simultáneamente en el móvil del cliente, en la tablet/móvil del camarero, en la pantalla de cocina y en el ordenador principal, la respuesta fue clara: tecnología web. No tendría sentido desarrollar cuatro aplicaciones nativas distintas para cubrir esos casos de uso. Una aplicación web bien diseñada es como tal multiplataforma: funciona en iOS, Android, Windows y Linux sin necesidad de compilaciones ni distribuciones separadas.

La elección del stack concreto es una mezcla de lo aprendido en el grado y otras tecnologías que considero necesarias para el proyecto y que he tenido que investigar. **Angular** para el frontend porque su arquitectura modular, el sistema de guards y la inyección de dependencias lo hacen especialmente adecuado para aplicaciones complejas con múltiples roles y rutas protegidas. **NestJS** para el backend porque su estructura por módulos (muy similar a Angular) facilita el mantenimiento y la escalabilidad, y porque integra de forma nativa tanto los controladores REST como los **WebSocket Gateways**, lo que permitió unificar la API y el tiempo real en el mismo proyecto. **Prisma** como ORM porque ofrece type safety completo, migraciones automáticas y un flujo de trabajo mucho más seguro que escribir SQL a mano. Y **PostgreSQL** como base de datos por su robustez, su soporte de tipos complejos como JSON (clave para el campo de pagos del pedido) y su madurez en entornos de producción.

Dificultades: Sincronización en tiempo real

La mayor dificultad del proyecto fue conseguir que todas las pantallas del sistema estuvieran siempre sincronizadas sin que el usuario tuviera que recargar la página. En un restaurante en funcionamiento, varios empleados pueden modificar el mismo pedido casi al mismo tiempo: el cliente añade un plato desde su móvil mientras el camarero ya está mirando la cuenta en el terminal y la cocina acaba de marcar otro plato como listo. Gestionar todo eso de forma coherente fue el problema central.

La solución pasó por adoptar una arquitectura orientada a eventos con WebSockets como canal de comunicación bidireccional entre el servidor y todos los clientes conectados. El backend actúa como “encargado”: cualquier operación que modifica el estado (un nuevo pedido, un plato listo, una cuenta solicitada, un cobro) se procesa en el servidor, se persiste en la base de datos y, solo entonces, se emite el evento correspondiente a todos los sockets suscritos. Ningún cliente actualiza su estado local sin confirmación del servidor.

Para organizar la distribución de eventos se implementó el sistema de salas de Socket.IO: cada dispositivo se une a las salas relevantes para su función (sala de cocina, sala de una mesa concreta, sala general). Un evento de nuevo pedido solo llega a la sala de cocina; un evento de cuenta solicitada llega a la sala general del dashboard. Esto simplificó la lógica de cada pantalla, que solo necesita escuchar los eventos de su sala.

Aprendizajes y valoración final

jPOS ha sido el proyecto más completo que he abordado hasta la fecha. Ha supuesto aprender tecnologías que no formaban parte del temario del ciclo (NestJS, Prisma, Socket.IO) y aplicarlas en un contexto realista con múltiples roles, múltiples pantallas y requisitos de consistencia estrictos. El hecho de conocer el dominio del problema desde dentro, por haberlo vivido trabajando en hostelería, me dio una perspectiva que creo que se nota en el resultado: las decisiones de diseño de la interfaz, los flujos de usuario y las prioridades del sistema están tomadas pensando en cómo funciona realmente un restaurante, no en cómo funciona un sistema POS tradicional.

En conjunto, estoy satisfecho con el resultado. jPOS es un sistema funcional, coherente y con potencial real de uso. Ha consolidado mis conocimientos full-stack, me ha enseñado a tomar decisiones de arquitectura y me ha demostrado que la mejor forma de aprender a programar bien es enfrentarse a un problema que de verdad importa resolver.

7. APÉNDICES



7.1. Comandos de instalación y arranque

Backend (jpos-backend/):

Instalar dependencias	npm install
Configurar base de datos	Editar DATABASE_URL y JWT_SECRET en el archivo .env
Ejecutar migraciones	npx prisma migrate deploy
Poblar datos iniciales	npx prisma db seed
Arrancar en desarrollo	npm run start:dev (con recarga automática)
Arrancar en producción	npm run build && npm run start:prod
Abrir Prisma Studio	npx prisma studio (interfaz visual de BD en puerto 5555)
Ejecutar tests unitarios	npm test
Cobertura de tests	npm run test:cov

Frontend (jpos-frontend/):

Instalar dependencias	npm install
Arrancar en desarrollo	npm start (puerto 4200, con recarga automática)
Compilar para producción	npm run build (salida en dist/)
Ejecutar tests	npm test

7.2. Variables de entorno (.env del backend)

DATABASE_URL	postgres://postgres:postgres@localhost:5432/jpos_db? schema=public
JWT_SECRET	Clave secreta para firmar los tokens JWT. Usar un valor aleatorio largo.
PORT	Puerto del servidor NestJS. Por defecto 3000 (opcional).

7.3. Rutas de la aplicación

Ruta	Componente	Guard	Roles mínimos
/login	LoginComponent	Ninguno (público)	Todos
/dashboard	Inicio (caja)	authGuard	Todos los roles
/mesas	Mesas	authGuard + rolGuard	ADMINISTRADOR, JEFE, EMPLEADO
/carta	Carta	authGuard + rolGuard	ADMINISTRADOR, JEFE
/reportes	Reportes	authGuard + rolGuard	ADMINISTRADOR
/configuracion	Configuración	authGuard + rolGuard	ADMINISTRADOR
/empleados	Usuarios	authGuard + rolGuard	ADMINISTRADOR
/terminal	Terminal TPV	authGuard + rolGuard	Todos los roles
/kds	Visor KDS	authGuard + rolGuard	Todos los roles

/qr:id	QrRedirect	Ninguno (público)	Cientes
/menu	Menú Cliente	Ninguno (público)	Cientes
/**	Redirección	—	Redirige a /login

7.4. Eventos WebSocket

Eventos del cliente hacia el servidor:

Evento	Payload	Descripción
unirse_sala	sala: string	Une al cliente a una sala de Socket.IO (ej: “cocina”).
enviar_comanda	{ mesald, sesionId, lineas[] }	El cliente o camarero envía un pedido. Crea o actualiza el pedido en BD.
notificar_cambio_mesa	mesald: string	Notifica a todos que el estado de una mesa ha cambiado (tras cobro o cancelación).
notificar_cambio_carta	—	Avisa a todos de que el catálogo ha sido modificado.
quitar_producto	{ mesald, lineald }	Elimina una línea de pedido de una mesa.
aplicar_descuento	{ mesald, tipo, valor }	Aplica un descuento (tipo: porcentaje / fijo, valor: number).
solicitar_cuenta	mesald: string	El cliente o camarero solicita que le traigan la cuenta.
marcar_linea_kds	{ pedidold, lineald, listo }	La cocina marca un plato como preparado o lo desmarca.

marcar_entregado_terminal	{ pedidold, lineald, entregado }	El camarero confirma que ha servido el plato al cliente.
cambiar_disponibilidad_kds	{ productold, disponible }	La cocina activa o desactiva la disponibilidad de un producto.

Eventos del servidor hacia los clientes:

Evento	Destino	Descripción
nuevo_pedido_cocina	Sala 'cocina'	Nuevo pedido listo para preparar. Solo lo reciben los dispositivos en la sala de cocina.
pedido_actualizado	Todos	Estado de un pedido ha cambiado. Todas las pantallas refrescan ese pedido.
mesas_actualizadas	Todos	El estado de alguna mesa ha cambiado. Todas las pantallas recargan la lista.
carta_actualizada	Todos	El catálogo ha cambiado. Todos los menús del cliente y terminales lo recargan.
cuenta_solicitada	Todos	{ mesald, numero }. El dashboard activa el aviso visual en la tarjeta de la mesa.
cuenta_cerrada	Todos	mesald. Todos los modales abiertos para esa mesa se cierran automáticamente.
plato_listo_notificacion	Todos	mesald. El terminal del camarero reproduce una notificación sonora.

7.5. Migraciones de base de datos

El historial de migraciones documenta la evolución del esquema a lo largo del desarrollo:

Migración	Cambios principales
init_jpos_schema	Esquema inicial: Usuario, Categoría, Producto, Mesa, Sesión, Pedido, LineaPedido.
anadir_configuracion	Añade el modelo Configuración con los datos del comercio.
anadir_turno_caja	Añade el modelo TurnoCaja para el control de caja.
update_pedido	Actualizaciones en el modelo Pedido (campo pagos JSON, estado CANCELADO).
add_config_opciones	Ampliación del modelo Configuración con nuevos campos opcionales.
add_listo_kds	Añade el campo “listo” a LineaPedido para el seguimiento en el KDS.
add_cuenta_solicitada	Añade el campo “cuentaSolicitada” a Mesa para la notificación de cuenta.

7.6. Documentación externa y referencias

Referencias a la documentación oficial de las tecnologías utilizadas durante el desarrollo del proyecto:

Tecnología	Referencia oficial	Uso principal
NestJS	https://docs.nestjs.com	Arquitectura del backend: módulos, controladores, servicios, guards, WebSockets.
Prisma ORM	https://www.prisma.io/docs	Definición del esquema, migraciones, consultas y Prisma Client.
PostgreSQL	https://www.postgresql.org/docs	Base de datos relacional: tipos de datos, índices, conexión.
Socket.IO	https://socket.io/docs/v4	Comunicación en tiempo real: eventos, salas, servidor y cliente.
Angular	https://angular.dev/docs	Framework SPA: componentes standalone, routing, guards, signals, i18n.
TailwindCSS	https://tailwindcss.com/docs	Sistema de diseño por utilidades: clases responsivas, colores, tipografía.
JWT (RFC 7519)	https://jwt.io/introduction	Estándar de tokens de autenticación sin estado.
bcrypt	https://www.npmjs.com/package/bcrypt	Cifrado de contraseñas con sal y múltiples rondas de hash.

ngx-translate	https://github.com/ngx-translate/core	Internacionalización del frontend Angular.
Chart.js	https://www.chartjs.org/docs	Librería de gráficas: dona, líneas, barras.
ng2-charts	https://valor-software.com/ng2-charts	Wrapper de Chart.js para Angular.
angularx-qrcode	https://www.npmjs.com/package/angularx-qrcode	Generación de códigos QR en componentes Angular.
RxJS	https://rxjs.dev/guide/overview	Programación reactiva: Observables, operadores, subjects.

GITHUB DEL PROYECTO:

jdelsan275/jPOS

jPOS - Sistema de gestión de punto de venta para hostelería. Proyecto fin de grado.



JesúsDS
GITHUB

 0
Contributors

 0
Issues

 0
Stars

 0
Forks

